מדריך לבעלי עסקים - הגדרה ב-5 דקות

מה זה בכלל?
דמיינו עובד שכל פעם שאתם מתקנים אותו - הוא כותב לעצמו פתק.
ובפעם הבאה? הוא קורא את כל הפתקים לפני שהוא מתחיל לעבוד.

זה בדיוק מה שהמערכת הזאת עושה.

קלוד קוד כבר יודע לשמור דברים בזיכרון. הבעיה? הוא לא עושה את זה לבד.

בדיוק בשביל זה אנחנו הולכים ליצור לולאת למידה.

תהליך אוטומטי שרץ ברקע, עובר על כל השיחות שלכם,
מזהה את הפעמים שתיקנתם אותו, ושומר את הלקחים.

התוצאה: ככל שעובר הזמן - קלוד טועה פחות ופחות.
והופך למפלצת AI אמיתית.

macOS - לכל מי שיש מקבוק: תעתיקו את הפרומוט למטה ותדביקו לקלוד קוד:

Set up an automatic self-learning system for Claude Code that runs once a day at 10:00 AM and learns from my corrections. This runs via my Claude Max subscription so there is no per-run cost. If the Mac is asleep or off at 10:00, LaunchAgent catch-up behavior will run it on next wake/startup.

PREREQUISITE CHECK BEFORE BUILDING ANYTHING:
- Run `claude --version` and confirm it works.
- Run `echo "hi" | claude -p -` and confirm it returns a response. If it asks for login, STOP and tell me to run `claude login` first before proceeding.
- Verify Python 3 is callable by running `python3 -c "print('ok')"`. It must return exactly "ok". If not, STOP and tell me to install Python 3.

HARD RULES BLOCK (this block must be COPIED VERBATIM into the daily prompt file that the bash script passes to `claude -p`. These rules apply to the LLM analyzing corrections on each daily run, NOT to you, the LLM building the install. Do not act on these rules; just embed them as text in the prompt file):

1. NEVER delete existing rules. Only strengthen, merge, or mark superseded with explicit reason. Deletion risks losing protection rules.
2. NEVER duplicate. Before adding a new rule, list files in the memory directory (path will be provided to you in this prompt) and check whether a similar rule exists. If yes, append context to that file instead of creating a new one.

3. ALWAYS include WHY in every rule. A rule without reason becomes dead weight in 90 days when the user no longer remembers the context.
4. SPECIFIC over vague. "Never use the phrase X in Hebrew SEO content on site.com" beats "Be careful with phrasing". Vague rules are unfixable.
5. Skip trivia. Single typos, exploratory what-ifs, one-time content edits are NOT corrections worth saving. Only persistent patterns warrant a saved rule.
6. Never infer intent from silence. Absence of correction is NOT approval. Only explicit positive words count as validation.
7. Max 5 NEW rules per run. Quality over quantity.
8. If the memory directory contains more than 100 .md files, BEFORE adding new ones, consolidate similar existing rules or note them as candidates for review in your output.
9. CONDITIONAL skills awareness: IF the directory ~/.claude/skills/ exists AND contains a skill whose name or description matches the topic of the correction, then append to that skill's SKILL.md under a "## Validated patterns" section instead of creating a new memory file. If the skills directory does not exist or contains nothing relevant, save to memory.

Here is what to build:

1. Python extraction script at ~/.nl-cron/scripts/extract_corrections.py.

OUTPUT FORMAT (CRITICAL): the script must write as its FIRST LINE the literal heading "# Self-Learn Extraction" followed by a space, an em-dash-free separator (use " | "), and the current ISO timestamp. Then a blank line. Then the extracted corrections.

The script finds all JSONL chat files in ~/.claude/projects/ modified in the last 24 hours (skip /subagents/ paths), parses each line as JSON looking for type "user" or "assistant", extracts text content (handle both string and array-of-objects format where each object has a text field), searches for correction patterns in Hebrew (לא נכון, טעות, תתקן, תשנה, למה עשית, לא ככה, שגוי, לא הבנת, עוד פעם, בדוק שוב, כתבנו במפורש, למה אתה לא עובר, לא הבנתי למה, לא צריך, תמחק, אל תעשה, סתם עשית, הכל נראה אותו דבר) and English (wrong, fix this, change this, that's not what I asked, don't do that, I already told you, we agreed, check again, redo, instead, I meant, you missed), ignores lines containing <ide_, <system-reminder, tool_result, <task-notification, <tool_use, and for each correction prints 2 messages before as context, the correction itself, and 1 message after. Truncate each message to 2000 chars, cap at 20 corrections.

PATTERN MATCHING (CRITICAL, reduces noise):
- Hebrew patterns use simple substring containment (Hebrew word boundaries differ; substring is acceptable).
- English patterns MUST use word-boundary regex matching, NOT substring containment: `re.compile(r"\b" + re.escape(pattern) + r"\b", re.IGNORECASE)`.
- The patterns "actually" and "no" are DELIBERATELY EXCLUDED because they appear too frequently in non-correction contexts.

CRITICAL output mechanism: the script accepts the output file path as `sys.argv[1]` and writes the markdown DIRECTLY to that file using `open(path, "w", encoding="utf-8")`. It

must NOT print to stdout. This avoids UnicodeEncodeError when launchd runs the script .with a minimal locale

Use os.path.expanduser for paths, encoding='utf-8' on EVERY open() call, handle malformed JSONL with try/except continue. If sys.argv[1] is missing, exit with code 2 and .a brief stderr message

Bash wrapper at ~/.nl-cron/scripts/self_learn.sh with the following EXECUTION .2 :ORDER

(PHASE 0: FAILURE NOTIFICATION (set up BEFORE anything that can fail
`Line 1: `#!/bin/bash -
(Line 2: `set -euo pipefail` (critical, without set -e the ERR trap below does nothing -
Set PATH including /usr/local/bin, /usr/bin, /bin, /sbin (critical: /sbin needed for -
/sbin/md5). For NVM node detection, use this exact pattern (do NOT use `sort -V`, which :(is not reliable on BSD/older macOS

""=NVM_BIN
if [-d "$HOME/.nvm/versions/node"]; then
for nvm_path in "$HOME/.nvm/versions/node"/*/bin; do
if [-x "$nvm_path/claude"]; then
":NVM_BIN="$nvm_path
break
fi
done
fi
"export PATH="${NVM_BIN}/usr/local/bin:/usr/bin:/bin:/sbin

This iterates installed nvm Node versions and prepends the first one that has `claude` installed. If no nvm directory exists, NVM_BIN stays empty and the system PATH is used .(as-is (fine for users with native claude installs

Define notify_failure function taking $1 (the message). It MUST: (a) run `osascript -e -
"display notification \"$1\" with title \"Claude Code Self-Learn\" sound name \"Basso\""`
interpolating the message text into the notification, and (b) append "$(date '+%Y-%m-%d
%H:%M:%S'): $1" to ~/Desktop/CLAUDE-LEARN-FAILED.txt
Install ERR trap IMMEDIATELY after notify_failure is defined: `trap 'notify_failure "Script -
`failed at line $LINENO"' ERR

Defining notify_failure + trap BEFORE Phase A means even mkdir failures in Phase A)
(.produce a notification

(PHASE A: RESOLVE MEMORY DIR + WORKING DIRECTORY (deterministic, lossless
Compute EXPECTED_PROJECT_ID by transforming $HOME: replace the leading slash -
with a dash, replace all remaining slashes with dashes. Example: /Users/john becomes

-Users-john. Use: `EXPECTED_PROJECT_ID=$(echo "$HOME" | sed 's|^/|-|' | sed .`("s|/|-|g
Set -
"EXPECTED_PROJECT_PATH="$HOME/.claude/projects/$EXPECTED_PROJECT_ID
Step (b): if "$EXPECTED_PROJECT_PATH/memory/MEMORY.md" exists, set -
MEMORY_DIR="$EXPECTED_PROJECT_PATH/memory" and
ORIGINAL_CWD="$HOME" (the project ID was derived from $HOME, so by construction .($HOME is the correct CWD; do NOT decode via sed
Step (c) FALLBACK: otherwise, find the most recently modified MEMORY.md across -
~/.claude/projects/*/memory/. If found, set MEMORY_DIR to its parent directory, then set
PROJECT_ID=$(basename "$(dirname "$MEMORY_DIR")"), then
ORIGINAL_CWD=$(echo "$PROJECT_ID" | sed 's|^-|/|' | sed 's|-|/|g'). Note: this
reverse-decoding is lossy for any path component that originally contained a literal hyphen
(e.g. a username like john-doe). If `cd "$ORIGINAL_CWD"` fails, fall back to `cd .`""$HOME
Step (d) FRESH USER: if no MEMORY.md exists anywhere, create -
"$EXPECTED_PROJECT_PATH/memory/" with `mkdir -p` and write a stub MEMORY.md
using this exact command: `printf '# Memory Index\n\n' >
"$EXPECTED_PROJECT_PATH/memory/MEMORY.md"`. Set
MEMORY_DIR="$EXPECTED_PROJECT_PATH/memory" and ."ORIGINAL_CWD="$HOME
After resolving: `cd "$ORIGINAL_CWD"` (the script's working directory determines what -
.project claude -p writes to). This is the critical CWD fix

PHASE B: ROBUSTNESS GUARDS
Every `find` command must end with `2>/dev/null || true` (avoids spurious alerts from -
.(iCloud / TimeMachine permission errors
Every `kill` command must end with `2>/dev/null || true` (killing an already-exited process -
.(must not trigger ERR
Commands that may legitimately return non-zero (`grep -c` with no matches) must end -
.(with `|| true` (NOT `|| echo 0` which produces concatenated output

PHASE C: SHORT-CIRCUIT CHECKS
`mkdir -p ~/.nl-cron/scripts ~/.nl-cron/logs ~/.nl-cron/state` -
Check for recent JSONL files: `find ~/.claude/projects/ -name "*.jsonl" -mmin -1440 -type -
.(f -not -path "*/subagents/*" 2>/dev/null | head -1`. If empty, log and exit 0 silently (no alert
Run the Python extractor: `python3 ~/.nl-cron/scripts/extract_corrections.py -
"$EXTRACT_PATH"` where EXTRACT_PATH=~/.nl-cron/state/corrections-extract.md
.If $EXTRACT_PATH does not exist, log and exit 0 silently -
Count corrections using this EXACT pattern (do NOT use `|| echo 0` which produces -
:(`"0\n0"` and breaks numeric comparison

(CORR_COUNT=$(grep -c "^### Correction" "$EXTRACT_PATH" 2>/dev/null || true
{CORR_COUNT=${CORR_COUNT:-0
if ["$CORR_COUNT" -eq 0]; then
"echo "$(date '+%Y-%m-%dT%H:%M:%S'): zero corrections" >> "$LOG_FILE
exit 0

fi

HASH SKIP: `EXTRACT_HASH=$(/sbin/md5 -q "$EXTRACT_PATH")` (use the absolute path to md5, not relying on PATH). Compare to ~/.nl-cron/state/.last_extract_hash if it exists. If identical, log "no new corrections since last run" and exit 0 silently. Do NOT update the hash file yet; only after a successful claude run.

PHASE D: BUILD PROMPT FILE

Build the prompt at ~/.nl-cron/state/prompt.md in this exact order:

i) Literal heading: "# Self-Learn Daily Run"

ii) A line: "Memory directory path: $MEMORY_DIR" (this is what hard-rule #2 references)

(iii) The HARD RULES BLOCK copied verbatim from earlier in this prompt (rules 1-9)

iv) Instructions to claude:

Analyze the corrections that follow -

For each genuine correction, decide whether to save as a new memory file or strengthen an existing one, following the HARD RULES -

Save new files in $MEMORY_DIR with names like feedback_<topic>.md (for behavioral rules), project_<name>.md (for project context), or reference_<system>.md (for pointers to external systems) -

The frontmatter format must be exactly this (each key on its own line, NOT comma-separated):

```
---
name: <short title>
description: <one line description>
type: feedback
---
```

(where type is one of: feedback, project, or reference)

Followed by a blank line, then the rule body.

Update $MEMORY_DIR/MEMORY.md by appending one line per new rule: `- [Title](filename.md) - one-line description` (use a regular hyphen, not an em-dash, in the description separator) -

Print "RULES EXTRACTED: N" followed by the list, then DONE. If nothing actionable, print "NO ACTIONABLE CORRECTIONS" then DONE -

v) Literal heading: "## Corrections to analyze":

vi) The contents of $EXTRACT_PATH appended)

PHASE E: INVOKE CLAUDE WITH WATCHDOG

Background claude: `claude -p - --dangerously-skip-permissions < "$PROMPT_FILE" > "$RUN_LOG" 2>&1 &`

IMMEDIATELY on the next line: `CLAUDE_PID=$!` (captures the PID of the backgrounded claude) -

Start watchdog: `( sleep 600; kill $CLAUDE_PID 2>/dev/null || true ) & WATCHDOG_PID=$!`

Wait pattern that captures exit code without tripping set -e: `wait $CLAUDE_PID -
`?$=2>/dev/null && EXIT_CODE=0 || EXIT_CODE
`Kill watchdog: `kill $WATCHDOG_PID 2>/dev/null || true -
If EXIT_CODE != 0 AND EXIT_CODE != 143 (143 = SIGTERM from watchdog timeout), -
.`"call `notify_failure "claude run failed with exit code $EXIT_CODE
If EXIT_CODE == 0, write the current $EXTRACT_HASH to -
~/.nl-cron/state/.last_extract_hash. (Intentionally NOT done for exit code 143: a watchdog-killed run leaves the hash unchanged, so the next run re-processes the same (.corrections
.Log a result line to ~/.nl-cron/logs/self_learn.log -

LaunchAgent plist at ~/Library/LaunchAgents/com.<actual-username>.self-learn.plist. .3
The filename MUST contain the username as a literal string obtained from running `whoami` at install time. Do not write the file at any path containing the characters $ or { or .`. Verify after creation by listing the file: the path must show the real username

:The plist must
Run the bash script daily at 10:00 via StartCalendarInterval with Hour=10, Minute=0 -
Stdout/stderr to ~/.nl-cron/logs/launchd_learn.log and launchd_learn.err.log -
EnvironmentVariables dict sets PATH (matching the wrapper) and HOME -
RunAtLoad=false -
Default StartCalendarInterval catch-up handles missed runs (fires once on next wake; -
(multiple missed days collapse to one run

INSTALLATION VERIFICATION (run all four; do NOT mark install complete until all .4
:(pass

VERIFICATION 1: extraction script produces UTF-8 output with the correct heading
`Run: `python3 ~/.nl-cron/scripts/extract_corrections.py /tmp/test-extract.md -
Check the file was actually created: `test -f /tmp/test-extract.md || { echo "FAIL: extractor -
`{ ;did not create output file"; exit 1
`Run: `head -3 /tmp/test-extract.md -
Expected: first line starts with "# Self-Learn Extraction"; file is valid UTF-8. If the heading -
.is missing, the Python script is wrong and must be fixed before continuing

VERIFICATION 2: failure path works
Temporarily inject `false` immediately after the ERR trap is set in Phase 0 -
`Run: `bash ~/.nl-cron/scripts/self_learn.sh -
Expected: macOS notification appears (text matching "Script failed at line N") AND -
~/Desktop/CLAUDE-LEARN-FAILED.txt has a new entry with current timestamp
`Remove the injected `false -

(VERIFICATION 3: CWD bug is actually fixed (THE most important check
Record baseline before the test: `BASELINE=$(ls ~/.claude/projects/-/memory/*.md -
`(2>/dev/null | wc -l
Run from root to simulate launchd's CWD=/: `cd / && bash -
`~/.nl-cron/scripts/self_learn.sh

Wait for the script to complete (it will either short-circuit silently if no corrections, or run - (claude for up to 10 minutes
AFTER it completes, run: `AFTER=$(ls ~/.claude/projects/-/memory/*.md 2>/dev/null | wc - `(-l
If $AFTER > $BASELINE, the install is BROKEN: claude wrote new files to the orphan - .path. Report and STOP. Do NOT mark install complete
.Also confirm new files (if any corrections existed) appear in $MEMORY_DIR -

VERIFICATION 4: launchd loads the plist
`launchctl load ~/Library/LaunchAgents/com.<actual-username>.self-learn.plist` -
(launchctl list | grep self-learn` should show the label with exit code 0 (or "-" if not yet run` -

:After all four verifications pass, print verbatim
Installation complete. The system runs daily at 10:00 AM. On the FIRST real failure," macOS may show a permission prompt asking to allow notifications. Approve it. Until ".approved, the desktop file CLAUDE-LEARN-FAILED.txt is your only alert channel

All paths must be dynamic using $HOME. The system must be SILENT when there is .nothing to learn but LOUD when something breaks

Windows לכל מי שיש ווינדוס: תעתיקו את הפרומוט למטה ותדביקו לקלוד קוד:

Set up an automatic self-learning system for Claude Code that runs once a day at 10:00 AM and learns from my corrections. I'm on Windows. This runs via my Claude Max subscription so there is no per-run cost. If the PC is off or asleep at 10:00, the scheduled .(task will run on next wake/logon (configure with StartWhenAvailable

:PREREQUISITE CHECK BEFORE BUILDING ANYTHING
.Run `claude --version` and confirm it works -
Run `Write-Output 'hi' | claude -p -` in PowerShell (NOT `"hi" | claude -p -` which passes - a PSObject, not stdin bytes). Confirm a response. If it asks for login, STOP and tell me to .run `claude login` first
Detect PowerShell version with `$PSVersionTable.PSVersion`. If it is 5.x, prefer `pwsh` - .(PowerShell 7+) if installed; otherwise proceed with 5.x but follow the rules below strictly
Verify Python is callable by RUNNING a trivial script: `python -c "print('ok')"` must return - exactly "ok". Fall back to `py -c "print('ok')"` if `python` is missing or returns empty (the Windows Store stub for `python` returns empty when not installed). If neither works, STOP .and tell me to install Python

:(ENCODING RULES (CRITICAL FOR HEBREW

The .ps1 file itself contains ZERO Hebrew strings. All Hebrew patterns live ONLY in the -
.Python script
Every PowerShell file write uses `-Encoding UTF8` explicitly: `Set-Content -Encoding -
UTF8`, `Out-File -Encoding UTF8`, `Add-Content -Encoding UTF8`. Never use `>`
.(redirection (which inherits PS5.1's default UTF-16
Python writes its outputs directly to files using `open(path, "w", encoding="utf-8")`. Never -
.pipe Python through PowerShell into a file

HARD RULES BLOCK (copy VERBATIM into the daily prompt file. These rules apply to
:(.the LLM analyzing corrections at runtime, NOT to you, the LLM building the install

NEVER delete existing rules. Only strengthen, merge, or mark superseded with explicit .1
.reason. Deletion risks losing protection rules
NEVER duplicate. Before adding a new rule, list files in the memory directory (path will .2
be provided to you in this prompt) and check whether a similar rule exists. If yes, append
.context to that file instead of creating a new one
ALWAYS include WHY in every rule. A rule without reason becomes dead weight in 90 .3
.days when the user no longer remembers the context
SPECIFIC over vague. "Never use the phrase X in Hebrew SEO content on site.com" .4
.beats "Be careful with phrasing". Vague rules are unfixable
Skip trivia. Single typos, exploratory what-ifs, one-time content edits are NOT .5
.corrections worth saving. Only persistent patterns warrant a saved rule
Never infer intent from silence. Absence of correction is NOT approval. Only explicit .6
.positive words count as validation
.Max 5 NEW rules per run. Quality over quantity .7
If the memory directory contains more than 100 .md files, BEFORE adding new ones, .8
.consolidate similar existing rules or note them as candidates for review in your output
CONDITIONAL skills awareness: IF the directory ~/.claude/skills/ exists AND contains a .9
skill whose name or description matches the topic of the correction, then append to that
skill's SKILL.md under a "## Validated patterns" section instead of creating a new memory
.file. If the skills directory does not exist or contains nothing relevant, save to memory

:Here is what to build

.Python extraction script at %USERPROFILE%\.nl-cron\scripts\extract_corrections.py .1

OUTPUT FORMAT (CRITICAL): the script must write as its FIRST LINE the literal heading
"# Self-Learn Extraction" followed by " | " and the current ISO timestamp. Then a blank
.line. Then the extracted corrections

The script finds all JSONL chat files in ~\.claude\projects\ modified in the last 24 hours
(skip \subagents\ paths), parses each line as JSON looking for type "user" or "assistant",
extracts text content (handle both string and array-of-objects format where each object
has a text field), searches for correction patterns in Hebrew (לא נכון, טעות, תתקן, תשנה, למה
עשית, לא ככה, שגוי, לא הבנת, עוד פעם, בדוק שוב, כתבנו במפורש, למה אתה לא עובר, לא הבנתי למה,
and English (wrong, fix this, change (לא צריך, תמחק, אל תעשה, סתם עשית, הכל נראה אותו דבר
this, that's not what I asked, don't do that, I already told you, we agreed, check again,

redo, instead, I meant, you missed), ignores lines containing <ide_, <system-reminder, tool_result, <task-notification, <tool_use, and for each correction prints 2 messages before as context, the correction itself, and 1 message after. Truncate each message to 2000 chars, cap at 20 corrections, write output as markdown DIRECTLY to a file path passed as .`sys.argv[1]`. Do NOT print to stdout

:(PATTERN MATCHING (CRITICAL
.Hebrew patterns use simple substring containment -
English patterns MUST use word-boundary regex matching: `re.compile(r"\b" + -
.`(re.escape(pattern) + r"\b", re.IGNORECASE
.The patterns "actually" and "no" are DELIBERATELY EXCLUDED -

Use os.path.expanduser("~") for paths, encoding='utf-8' on EVERY open() call, handle malformed JSONL with try/except continue. If sys.argv[1] is missing, exit with code 2 and .a brief stderr message

PowerShell wrapper at %USERPROFILE%\.nl-cron\scripts\self_learn.ps1 with this .2 :EXECUTION ORDER

PHASE 0: ENCODING, ERROR HANDLING, NOTIFY-FAILURE
:Top of file -
`Console]::OutputEncoding = [System.Text.Encoding]::UTF8]`
`OutputEncoding = [System.Text.Encoding]::UTF8$`
`'ErrorActionPreference = 'Stop$`
Define Notify-Failure function taking $message. MUST: (a) append "$(Get-Date -Format - 'yyyy-MM-dd HH:mm:ss'): $message" to "$env:USERPROFILE\Desktop\CLAUDE-LEARN-FAILED.txt" using `Add-Content -Encoding UTF8`, and (b) try to show a toast via `New-BurntToastNotification -Text "Claude Code", $message` wrapped in try/catch with empty catch. Do NOT auto-install .BurntToast
:All script logic from this point on is inside ONE outer try/catch -
`{ try { ... } catch { Notify-Failure $_.Exception.Message; throw`
Inside try-block code throws `throw "descriptive message"` on internal failures. Only the - .outer catch calls Notify-Failure

(PHASE A: RESOLVE MEMORY DIR + WORKING DIRECTORY (deterministic, lossless
Compute EXPECTED_PROJECT_ID by transforming $env:USERPROFILE: strip the - drive letter and colon (e.g. C: removed), replace all backslashes with dashes, prepend a :dash. Example: C:\Users\john becomes -Users-john. Use
`('-' ,'\\' expectedId = '-' + ($env:USERPROFILE -replace '^[A-Za-z]:', '' -replace$`
VALIDATE THE ENCODING by listing what actually exists at - `$env:USERPROFILE\.claude\projects\`. If $expectedId is NOT among the directory names there AND other project dirs exist, log a warning that the encoding heuristic may .differ from Claude Code's actual encoding; still proceed with the fallback
"Set $expectedPath = "$env:USERPROFILE\.claude\projects\$expectedId -
Step (b): if `Test-Path "$expectedPath\memory\MEMORY.md"`, set $memoryDir = - "$expectedPath\memory" and $workingDir = $env:USERPROFILE. (The expected ID was

derived from $env:USERPROFILE so by construction this is the correct working directory; (.do NOT decode any path
Step (c) FRESH USER OR NO MATCH: otherwise (whether the user has zero projects, - or has projects but none matches $expectedId), CREATE a new project at the expected path. Run `New-Item -ItemType Directory -Force -Path "$expectedPath\memory"`, then :write the stub using
Set-Content -Path "$expectedPath\memory\MEMORY.md" -Encoding UTF8 -Value "#` `"Memory Index`r`n
Use a real CRLF newline via backtick-r backtick-n inside the string, not the literal) characters \r\n.) Set $memoryDir = "$expectedPath\memory" and $workingDir = .$env:USERPROFILE
IMPORTANT: do NOT attempt to find "most recently modified MEMORY.md" across other - projects on Windows. The Split-Path reverse approach used on macOS does not work cleanly here because the relationship between $env:USERPROFILE and Claude Code's Windows project ID encoding is heuristic. The safer behavior on Windows is: if the expected project does not exist, create it. This guarantees $workingDir = .$env:USERPROFILE always matches the resolved memory dir
Set-Location $workingDir` (working directory determines what project claude -p writes` - .(to

PHASE B: ROBUSTNESS (no-op on Windows; error handling is via $ErrorActionPreference = 'Stop' set in Phase 0 + outer try/catch. This phase label exists (.only to keep ordering aligned with the macOS version

(PHASE C: SHORT-CIRCUIT CHECKS (inside the try
Create dirs: `New-Item -ItemType Directory -Force -Path - "$env:USERPROFILE\.nl-cron\scripts", "$env:USERPROFILE\.nl-cron\logs", `"$env:USERPROFILE\.nl-cron\state" | Out-Null
Check for JSONL files modified in last 24h: `$recent = Get-ChildItem -Path - "$env:USERPROFILE\.claude\projects\" -Recurse -Filter *.jsonl -ErrorAction SilentlyContinue | Where-Object { $_.LastWriteTime -gt (Get-Date).AddHours(-24) -and $_.FullName -notmatch '\\subagents\\' } | Select-Object -First 1`. If $null, log and exit .silently
Run the Python extractor: try `python - "$env:USERPROFILE\.nl-cron\scripts\extract_corrections.py" "$extractPath"` first; if exit code non-zero, try `py` with same args. If both fail, `throw "Python not found or extractor .`"failed
.If $extractPath does not exist or is empty, log and exit silently -
HASH SKIP: `$extractHash = (Get-FileHash $extractPath -Algorithm SHA256).Hash`. - Compare to "$env:USERPROFILE\.nl-cron\state\.last_extract_hash" if it exists. If identical, log "no new corrections since last run" and exit silently. Do NOT save the new hash yet; .only after a successful claude run

PHASE D: BUILD PROMPT FILE
Build the prompt at "$env:USERPROFILE\.nl-cron\state\prompt.md" using `Set-Content -Encoding UTF8` for the first write, then `Add-Content -Encoding UTF8` for subsequent :(appends. Structure (in order

"i) "# Self-Learn Daily Run)
"ii) "Memory directory path: $memoryDir)
(iii) The HARD RULES BLOCK copied verbatim from earlier (rules 1-9)
:iv) Instructions to claude)
Analyze the corrections that follow -
For each genuine correction, decide whether to save as a new memory file or -
strengthen an existing one, following the HARD RULES
Save new files in $memoryDir with names like feedback_<topic>.md, -
project_<name>.md, or reference_<system>.md
The frontmatter format must be exactly this (each key on its own line, NOT -
:(comma-separated

<name: <short title
<description: <one line description
type: feedback

(where type is one of: feedback, project, or reference)
.Followed by a blank line, then the rule body
Update $memoryDir\MEMORY.md by appending one line per new rule: `- -
([Title](filename.md) - one-line description` (use a regular hyphen as the separator
Print "RULES EXTRACTED: N" followed by the list, then DONE. If nothing -
.actionable, print "NO ACTIONABLE CORRECTIONS" then DONE
":v) "## Corrections to analyze)
vi) The contents of $extractPath appended via `Get-Content $extractPath -Raw)
`-Encoding UTF8

PHASE E: LOCATE AND INVOKE CLAUDE (write a .bat wrapper to avoid both .NET
(pipe-buffer deadlock AND PowerShell quoting fragility for paths with spaces
Locate claude executable. Try `Get-Command claude` first. If not found, check in order: -
`$env:APPDATA\npm\claude.cmd`,
`$env:LOCALAPPDATA\Programs\Anthropic\Claude\claude.exe`,
`$env:LOCALAPPDATA\Programs\claude\claude.exe`,
`$env:ProgramFiles\Anthropic\Claude\claude.exe`,
`$env:ProgramFiles\Anthropic\claude.exe`. If none found, `throw "claude executable not
.`"found

INVOCATION (CRITICAL: this is the most fragile section. Two failure modes must both be
avoided: (1) [System.Diagnostics.Process] with RedirectStandardOutput hangs when
claude writes more than ~64KB to stdout (.NET pipe buffer), (2) cmd /c via Start-Process
-ArgumentList re-quotes when paths contain spaces like "C:\Users\John Doe\", producing
.wrong command lines

The robust approach is to write a temporary .bat file that contains the literal command
(cmd handles its own quoting natively, the OS handles the I/O pipes), then execute the
:(..bat. Both failure modes are eliminated

Write a one-shot batch file with the literal command. cmd handles paths with #

.spaces correctly when they are inside double quotes in a .bat file #
"batPath = "$env:USERPROFILE\.nl-cron\state\run_claude.bat$
"@ = batContent$
echo off@
claudePath" -p - --dangerously-skip-permissions < "$promptPath" > "$runLogPath" 2>&1$"
@"
Set-Content -Path $batPath -Value $batContent -Encoding ASCII

.Spawn the .bat. No quoting layer here because Start-Process gets a single file path #
proc = Start-Process -FilePath $batPath -PassThru -NoNewWindow$
(exited = $proc.WaitForExit(600000$
} (if (-not $exited
()proc.Kill$
()proc.WaitForExit$
"throw "Claude run timed out after 10 minutes
{
} (if ($proc.ExitCode -ne 0
"(throw "Claude exited with code $($proc.ExitCode
{

NOTE on the .bat encoding: written with `-Encoding ASCII` because cmd.exe parses .bat files in the system's OEM code page and chokes on BOM-prefixed UTF-8. The .bat only contains paths and ASCII flags; no Hebrew/unicode goes into it. If a user has unicode in their USERPROFILE path (rare, e.g. accented characters), this approach may fail and the .user will need to relocate their profile or report it

On success, save the hash: `Set-Content -Path -
"$env:USERPROFILE\.nl-cron\state\.last_extract_hash" -Value $extractHash -Encoding
`UTF8
Log result line: `Add-Content -Path "$env:USERPROFILE\.nl-cron\logs\self_learn.log" -
-Value "$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss'): exit=0 hash=$extractHash"
`-Encoding UTF8

:Windows Scheduled Task named ClaudeCode-SelfLearn .3
Action: powershell.exe (or pwsh.exe if PS7+ detected). The arguments must be a list -
where the -File argument is the LITERAL EXPANDED path on disk, not a placeholder.
:Build
`"ps1Path = "$env:USERPROFILE\.nl-cron\scripts\self_learn.ps1$`
action = New-ScheduledTaskAction -Execute 'powershell.exe' -Argument$`
`""`"-ExecutionPolicy Bypass -NoProfile -File `"$ps1Path
`Trigger: `New-ScheduledTaskTrigger -Daily -At 10:00AM -
Settings: `New-ScheduledTaskSettingsSet -AllowStartIfOnBatteries -
`-DontStopIfGoingOnBatteries -StartWhenAvailable
Principal: `New-ScheduledTaskPrincipal -UserId $env:USERNAME -LogonType -
Interactive -RunLevel Limited`. Without an explicit Principal, the task runs as SYSTEM
.%which has no access to %USERPROFILE

Register: `Register-ScheduledTask -TaskName "ClaudeCode-SelfLearn" -Action $action -
`-Trigger $trigger -Settings $settings -Principal $principal

:(INSTALLATION VERIFICATION (run all; do NOT mark install complete until all pass .4

VERIFICATION 1: extraction script produces UTF-8 output with the correct heading
python "$env:USERPROFILE\.nl-cron\scripts\extract_corrections.py"` -
`""$env:TEMP\test-extract.md
Check the file was actually created: `if (-not (Test-Path "$env:TEMP\test-extract.md")) { -
`{ "throw "FAIL: extractor did not create output file
`Get-Content "$env:TEMP\test-extract.md" -Encoding UTF8 | Select-Object -First 3` -
Expected: first line starts with "# Self-Learn Extraction"; file is valid UTF-8. If the heading -
.is missing, the Python script is wrong and must be fixed before continuing

VERIFICATION 2: failure path works
Temporarily inject `throw "test failure"` immediately after Notify-Failure is defined and the -
try block opens
Run: `powershell -ExecutionPolicy Bypass -File -
`""$env:USERPROFILE\.nl-cron\scripts\self_learn.ps1
Expected: ~/Desktop/CLAUDE-LEARN-FAILED.txt has EXACTLY ONE new entry with -
(the current timestamp (not two
Remove the test throw -

VERIFICATION 3: CWD bug is actually fixed (THE most important check — must wait for
(actual completion
BEFORE STARTING: warn the user verbatim: "Verification 3 runs the daily script -
synchronously. If there are corrections from the last 24 hours and the hash has changed,
this will invoke claude -p which can take 1-10 minutes. Do not interrupt. If there are no
".recent corrections, this finishes in seconds
Record baseline: `$baseline = (Get-ChildItem -
"$env:USERPROFILE\.claude\projects\-\memory\" -Filter *.md -ErrorAction
`SilentlyContinue).Count
Run the script DIRECTLY and SYNCHRONOUSLY (do NOT use Start-ScheduledTask -
:(which races against actual completion
powershell -ExecutionPolicy Bypass -NoProfile -File`
`""$env:USERPROFILE\.nl-cron\scripts\self_learn.ps1
.This will return when the script finishes -
After it returns, check: `$after = (Get-ChildItem -
"$env:USERPROFILE\.claude\projects\-\memory\" -Filter *.md -ErrorAction
`SilentlyContinue).Count
.If $after > $baseline, the install is BROKEN. Report and STOP -
Also check: `Get-ChildItem $memoryDir -Filter *.md | Where-Object { $_.LastWriteTime -
-gt (Get-Date).AddMinutes(-15) }`. New rules (if any corrections were processed) should
.appear here

VERIFICATION 4: task registration

Get-ScheduledTask -TaskName ClaudeCode-SelfLearn | Select-Object State, Principal`.` - Acceptable States: "Ready" or "Queued" or "Running" (any of these means the task is registered and will fire on schedule). REJECT only "Disabled" or empty. Principal must contain the actual username (NOT "SYSTEM"). If the State is "Disabled", run .`Enable-ScheduledTask -TaskName ClaudeCode-SelfLearn` and re-check

:After all four verifications pass, print verbatim
Installation complete. With LogonType Interactive, the daily run requires you to be logged" in. If your PC is locked or you're logged out at 10:00 AM, the task will run as soon as you next log in. The desktop file CLAUDE-LEARN-FAILED.txt remains your reliable alert ".channel

All paths must be dynamic using $env:USERPROFILE. The system must be SILENT .when there is nothing to learn but LOUD when something breaks

שאלות נפוצות

❓ זה עולה כסף נוסף?
הלופ צורך טוקנים מהמנוי שלכם. כל ריצה ≈ שיחה קצרה. עלות זניחה.

❓ זה מאט את העבודה?
לא.

❓ מה אם קלוד ילמד משהו לא נכון?
זה יכול לקרות. לכן הוא אף פעם לא מוחק כללים ישנים, מוסיף מקסימום 5 בריצה, ואתם יכולים תמיד לבקש show me lessons.md ולערוך ידנית.

❓ זה עובד בעברית?
כן. הסורק מזהה תיקונים גם בעברית וגם באנגלית.

❓ אפשר לכבות?
בטח. תגידו לו "תעצור את המשימה המתוזמנת שלי" והוא יעצור.

❓ זה עובד על כל פרויקט?
כן. הפרומפט גנרי קלוד מזהה לבד את מבנה הפרויקט ואת קבצי הזיכרון.

❓ כמה זמן עד שאראה חוקים?
ייתכן ש-3-7 ימים יעברו בלי שום חוק חדש זה תקין.

המערכת מוסיפה רק חוקים חדשים שעוד לא קיימים אם תיקנתם משהו שכבר נמצא ב-MEMORY.md, היא תזהה ולא תכפיל.
ציפיה ריאלית: 3-8 חוקים חדשים בחודש.

❓ איפה אני רואה מה קלוד למד?

פשוט תגידו לו: "תראה לי את MEMORY.md שלי"
או: "מה החוקים החדשים שלמדת השבוע?"
הוא יציג רשימה. אתם יכולים לערוך/למחוק כל חוק שלא רלוונטי.